BASES DE DATOS II

Trabajo Práctico

ORM con Hibernate y anotaciones JPA

Este primer Trabajo Práctico estará basado en la implementación de una aplicación Java orientada al estudio de la persistencia, tanto en términos conceptuales, como en el uso de tecnología en particular. La aplicación consistirá en una arquitectura multicapa clásica, debiéndose implementar una capa de lógica de aplicación (servicios) y otra de acceso a datos (repositorios) subyacentes.

La cátedra entregará un proyecto modelo implementado con Maven y el framework Spring y una configuración de Hibernate inicial, así como una serie de test cases que sirven de guía para resolver los diferentes puntos. Como resultado, se obtendrá la implementación de todo lo necesario para que tanto los test cases pasen como para que la aplicación quede completa. En términos concretos, al correr el comando

mvn clean install

desde la línea de comandos debe obtenerse un build exitoso en donde pasen todos los tests.

El proyecto base se encuentra disponible en Github. No realizar un fork del proyecto, ya que esto genera un repositorio público, mientras que la entrega debe realizarse a través de un repositorio privado.

Link del repositorio base: https://github.com/fedediclaudio/tp-bbdd2-tours

Descripción del dominio

Se quiere generar una app de viajes turísticos para un emprendimiento local.

Una vez que una persona se ha registrado en el sistema de Turismo, puede elegir alguno de los varios recorridos diarios que la empresa ofrece y comprarlo.

Cada recorrido (*Route*) tiene su nombre, que es representativo del tramo que visita, el valor del mismo, el total en km del recorrido y pueden visualizarse además cada una de las paradas (*Stop*) que componen el recorrido. Todos los recorridos se realizan una vez por día. Asimismo, la aplicación registra comentarios y valoraciones (estrellas) de otros usuarios que han realizado el mismo recorrido.

Dada la cantidad de personas que solicitan los recorridos, la empresa ha tenido que limitar cada recorrido a un tope máximo de personas. Esa cantidad no es general sino que se define para cada uno y se deben vender solo mientras el cupo lo permita.

Adicionalmente, se conoce un grupo de choferes y guías turísticos que participan del recorrido.

A modo de ejemplo, podemos pensar en el recorrido "Tour tradicional de Bs. As." que cuenta con paradas en distintos lugares emblemáticos de la ciudad de Buenos Aires como Caminito en La

Boca, San Telmo, Obelisco, lago de Palermo y Recoleta; o en "Tour del Delta" que incluye a San Isidro, el Delta y también al Obelisco.

Una vez que se ha registrado la compra de un usuario para un recorrido determinado (*Purchase*), se le habilita la posibilidad de agregar comentarios y puntaje al recorrido realizado (*Review*) de forma tal que luego puedan usarlo en promociones del emprendimiento y poder captar así a más público interesado en los mismos.

Se ha pensado que los usuarios (*User*) se identifiquen en la aplicación mediante usuario y contraseña, y otros datos que pueden resultar útiles para posteriores ofertas de tours que la empresa realice. Por eso se les solicitan datos personales y de contacto como el nombre, un número de celular, un email, edad y fecha de nacimiento.

Los choferes (*DriverUser*) y guías turísticos (*TourGuideUser*) también son usuarios del sistema, por lo que se registran los mismos datos que los usuarios, pero además cuentan con una serie de antecedentes que la empresa quiere mantener: del chofer, de trabajos anteriores; del guia turistico, sobre su formación en turismo. Cabe mencionar que los choferes y los guías no están afectados a un recorrido en particular, sino que pueden participar de varios según la demanda y la necesidad de la empresa. En fechas particulares, por ejemplo, hay mayor demanda por los tours de la zona del Delta.

Durante todo el recorrido se ofrecen diversos servicios (*Service*) que un usuario puede contratar. De los servicios se conoce su nombre, su valor, una descripción y el proveedor que lo brinda. Por ejemplo, la confección de algún recuerdo (un mate pintado artesanalmente, una remera o una foto en un lugar determinado).

De cada proveedor (*Supplier*) se conocen los servicios que ofrece, la razón social y un número de habilitación provincial.

Cada ítem de servicio aparece en la compra junto con el recorrido y la cantidad solicitada del mismo (*ItemService*).

La compra final contendrá: el recorrido elegido y todos los servicios solicitados a lo largo del mismo (para que el usuario pueda tener control de lo abonado). Además, sobre cada compra se registra un código interno y la fecha.

> ***El objetivo del TP es agregar el mapeo JPA/Hibernate, construir la capa de acceso a datos usando Hibernate y responder las preguntas del TP.***
>
> ***Se entregará una implementación base en Java sin capa de persistencia (ver introducción). Se proporcionarán tests para verificar cada implementación.***

Diagrama de Clases del Modelo

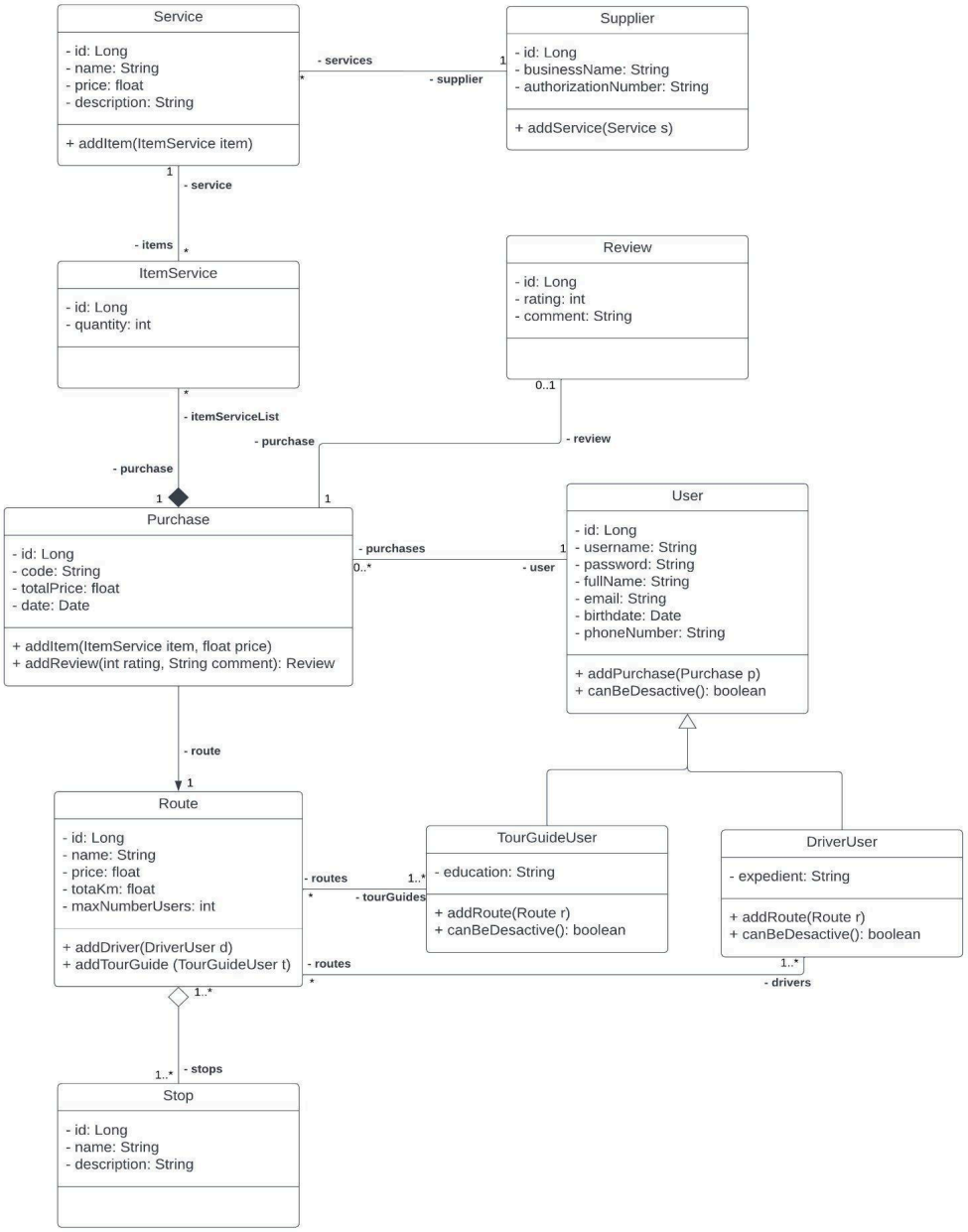

Figura 1 - Diagrama de clases del sistema de gestión de tours. Consultar en todo momento al resolver los ejercicios.

Descripción de las entidades:

Entidad	Descripción	Relaciones principales
Service	Servicio ofrecido con nombre y precio	Pertenece a un Supplier; referenciado por ItemService
Supplier	Proveedor que ofrece servicios	Tiene muchos Service
ItemService	Item de servicio en una compra (cantidad)	Referencia a Service; pertenece a Purchase
Purchase	Compra de un usuario sobre una ruta	Tiene ItemService, User, Route y Review (0..1)
Review	Reseña con rating y comentario	Asociada a una Purchase (0..1)
User	Usuario base del sistema	Subclasificado en DriverUser y TourGuideUser
DriverUser	Conductor (subclase de User)	Asignado a una o más Route como driver
TourGuideUser	Guia turistico (subclase de User)	Asignado a una o más Route como tourGuide
Route	Ruta con nombre, precio y capacidad	Tiene Stop (1..*), drivers y tourGuides
Stop	Parada de una ruta	Composicion con Route (1..*)

Seccion 1 - Introduccion a los ORM

Esta sección presenta los fundamentos conceptuales del mapeo objeto-relacional (ORM), con foco en JPA como especificación y Hibernate como implementación. El objetivo es comprender el problema que resuelven y las abstracciones que ofrecen.

1.1 El problema del ORM

Los ORM surgen para salvar la brecha estructural entre el paradigma orientado a objetos y el paradigma relacional. Responder las siguientes preguntas:

1. ¿Qué problema concreto resuelve un ORM? Describir al menos 3 (tres) inconvenientes que aparecen al intentar persistir objetos directamente con JDBC puro.
2. El modelo relacional y el modelo OO presentan tensiones conocidas como *impedance mismatch*. Identificar cómo se manifiesta cada una en el modelo dado:
   a) Identidad: cómo es posible identificar un objeto Purchase en Java vs. en la base de datos?
   b) Relaciones: cómo se navega de una Purchase a sus ItemService en Java vs. en SQL?
   c) Herencia: ¿cómo podría representarse la jerarquía User/DriverUser/TourGuideUser en una tabla relacional?
   d) Ciclos de referencia: ¿existe algún ciclo en el modelo? ¿Cómo impacta en la persistencia?
3. Describir las ventajas y desventajas concretas de usar un ORM en un proyecto como este.

1.2 JPA e Hibernate

JPA define el contrato de persistencia estándar en Java. Hibernate es una implementación que utiliza esta especificación. Entender la diferencia es fundamental para poder trabajar con ambas correctamente.

4. ¿Qué es JPA? ¿Qué nos permite definir? ¿Por qué se dice que es una especificación y no una implementación?
5. ¿Cuál es la relación entre JPA e Hibernate? ¿Puede usarse JPA sin Hibernate o Hibernate sin JPA?
6. ¿Qué es la SessionFactory? ¿Qué patrón de diseño implementa? Justificar por qué se crea una sola instancia durante todo el ciclo de vida de la aplicación y no una por operación.
7. La SessionFactory ofrece dos formas de obtener una Session: openSession() y getCurrentSession(). Responder:
   a) ¿Cuál es la diferencia entre ambos métodos? ¿Qué ocurre con el ciclo de vida de la Session en cada caso?
   b) ¿Qué condición debe cumplirse para poder usar getCurrentSession()? ¿Que configuración requiere en Hibernate?

c) ¿Quién es responsable de cerrar la Session cuando se usa getCurrentSession()? ¿Y cuando se usa openSession()?

d) ¿Cuál de los dos métodos resulta más adecuado para usar en los repositorios de esta práctica y por qué?

8. Completar la tabla comparativa entre JPA e Hibernate:

Aspecto	JPA	Hibernate
Tipo		
Define interfaces y anotaciones		
Proporciona implementación concretas		
Genera SQL a partir de consultas JPQL/HQL		
Maneja el Persistence Context		

1.3 Ciclo de vida de las entidades

Hibernate gestiona los objetos de dominio a través de un ciclo de vida bien definido. Conocerlo es esencial para entender cuándo y cómo se persisten los cambios.

9. Describir los cuatro estados posibles de una entidad en Hibernate (transient, managed, detached, removed) e indicar qué operación dispara cada transición entre ellos.
10. Describir el ciclo de vida completo de una entidad persistente, tome como ejemplo un objeto de la clase Purchase: desde que se instancia con new, pasando por su persistencia, hasta que se elimina. Indicar explícitamente en qué estado se encuentra el objeto en cada paso.
11. Investigue sobre los métodos de Session: session.save(), session.persist(), session.merge() y session.saveOrUpdate(). ¿Qué permite hacer cada uno y cuál es la diferencia entre ellos? Indicar en qué estado debe estar un objeto para usar cada uno correctamente.

Sección 2 - Mapeo de Entidades

En esta sección se implementa el mapeo de todas las clases del modelo a tablas relacionales usando anotaciones JPA. El objetivo es tener al final de la sección la totalidad de las entidades correctamente mapeadas y verificadas con los tests provistos.

2.1 Mapeo de una entidad simple: Service

Comenzamos por la entidad Service, que no tiene jerarquía ni relaciones complejas, para introducir las anotaciones fundamentales de JPA antes de avanzar hacia los casos más complejos del modelo.

12. ¿Cuál es el conjunto mínimo de anotaciones que debe tener una clase para ser persistente con JPA?
13. ¿Qué significa que JPA use persistencia por alcance (*persistence by reachability*)? ¿Qué consecuencia tiene si un objeto referenciado no está todavía persistido?
14. ¿Qué diferencia hay entre las estrategias IDENTITY, SEQUENCE y TABLE para la generación de IDs? ¿Cuál tiene mejor rendimiento en inserciones masivas y por qué?
15. Implementar el mapeo completo de la entidad Service según el diagrama. La implementación debe incluir:
    e) Clave primaria con estrategia de generación automática. Elegir entre IDENTITY, SEQUENCE o TABLE y justificar la elección.
    f) Atributos: name (no nulo, max. 100 caracteres), description (opcional), price (no nulo).
    g) Al menos una restricción de unicidad a nivel de columna.

2.2 Relaciones entre entidades

Una vez mapeadas las primeras entidades, el siguiente paso es incorporar las relaciones del modelo. A continuación se trabajan las distintas formas de mapear asociaciones en JPA y se pide implementar todas las entidades restantes (salvo la jerarquía que se trata en la sección siguiente).

16. Para la relación Purchase -> ItemService (composición uno-a-muchos):
    h) ¿Qué anotaciones se necesitan en cada lado?
    i) ¿Qué columna o tabla aparece en la base de datos para representar esta relación?
    j) ¿Qué es mappedBy y en qué lado de la relación va? ¿Qué ocurre si se omite en ambos lados?
    k) ¿Es esta relación bidireccional o unidireccional según el diagrama? ¿Cómo se refleja en el código Java?
17. Para las relaciones Route <-> DriverUser y Route <-> TourGuideUser (muchos-a-muchos):
    l) ¿Qué anotaciones se usan?
    m) ¿Qué tabla adicional genera JPA? ¿Qué columnas tiene? Definirla explícitamente usando @JoinTable.
    n) ¿Pueden ambas relaciones compartir la misma tabla join? ¿Por qué?
18. La relación Purchase -> Review es opcional (0..1). Implementar el mapeo de ambos lados. ¿Cómo se representa la opcionalidad en JPA?
19. ItemService referencia a Service (muchos-a-uno). Analizar el diagrama: ¿es navegable esta relación desde Service hacia ItemService? Justificar si conviene hacerla bidireccional o no.
20. Implementar el mapeo completo de las siguientes entidades con todas sus relaciones, siguiendo las anotaciones y decisiones discutidas: Supplier, Purchase, ItemService, Route, Stop y Review. Para cada relación bidireccional, incluir las anotaciones en ambos lados.

> ***En relaciones bidireccionales, mantener ambos lados sincronizados es responsabilidad del desarrollador. Si se agrega un ItemService a una Purchase, también se debe setear la referencia purchase en el ItemService; Hibernate no lo hace automáticamente.***

2.3 Fetch Type: LAZY vs. EAGER

El tipo de carga de una relación define si los objetos asociados se traen de la base de datos en el momento en que se carga la entidad o solo cuando se accede a ellos por primera vez. Esta decisión impacta directamente en la performance y en los posibles errores de la aplicación.

21. ¿Para qué sirve la propiedad fetch en las anotaciones de relación? ¿Cuáles son los valores posibles? ¿Cuál es el valor por defecto para @OneToMany, para @ManyToOne y para @ManyToMany?
22. Describir ventajas y desventajas concretas de EAGER y LAZY, en términos de performance de acceso como de espacio en memoria. ¿Por qué configurar EAGER en todas las relaciones suele ser una mala idea en aplicaciones reales?
23. Para cada relación del modelo, elegir el FetchType más adecuado y justificar. Luego implemente su decisión en el proyecto tours.

Relación	Tipo de anotación	FetchType elegido	Justificación
Purchase -> User	@ManyToOne		
Purchase -> Route	@ManyToOne		
Purchase -> itemServiceList	@OneToMany		
Purchase -> Review	@OneToOne		
ItemService -> Service	@ManyToOne		
Route -> stops	@OneToMany		
Route -> drivers (DriverUser)	@ManyToMany		
Route -> tourGuides (TourGuideUser)	@ManyToMany		
User -> purchases	@OneToMany		
Service -> supplier	@ManyToOne		

24. ¿Cómo podría producirse una LazyInitializationException en el modelo? Investigue de qué representa esta excepción y escribir un escenario concreto explicando al menos formas de resolverlo sin cambiar el FetchType a EAGER.

2.4 Operaciones en cascada

Las operaciones en cascada determinan si una operación realizada sobre una entidad (persistir, eliminar, etc.) debe propagarse automáticamente a las entidades que referencia. Configurarlas correctamente es fundamental para mantener la consistencia del modelo.

25. Enumerar todos los valores de CascadeType y explicar qué operación propaga cada uno sobre las entidades relacionadas.
26. ¿Cuál es el comportamiento por defecto cuando no se define cascade? ¿Cuál es la finalidad general de CASCADE? Proponga un caso del modelo donde definir un CASCADE inadecuado podría traer problemas a la consistencia de la base de datos.
27. ¿Cuál es la diferencia entre cascade = REMOVE y orphanRemoval = true? ¿Pueden usarse conjuntamente? Ejemplificar con el par Purchase -> ItemService.
28. Para la relación Purchase -> ItemService (composición):
    o) ¿Qué tipos de cascade configurarías? Justificar cada uno.
    p) ¿Usarias orphanRemoval? ¿Por qué?
    q) Describir qué ocurre a nivel de base de datos cuando se elimina un ItemService de la lista de una Purchase y Hibernate realiza el flush.
29. Para la relación Purchase -> Review:
    r) ¿Qué cascades configurarías?
    s) Si se elimina una Purchase, ¿debería eliminarse también su Review? Justificar desde el modelo de negocio.
30. Para la relación Supplier -> Service:
    t) ¿Qué cascades tienen sentido?
    u) Si se elimina un Supplier, ¿qué debería ocurrir con sus Service? ¿Y con las Purchase que los contienen a través de ItemService?
31. ¿Por que CascadeType.REMOVE en una relación @ManyToMany (por ejemplo Route <-> DriverUser) suele ser peligroso? Describir un escenario donde su uso cause pérdida no deseada de datos.
32. Implemente las operaciones en cascada adecuada para todas las relaciones del modelo tours.

2.5 Jerarquía de herencia: User, DriverUser y TourGuideUser

El modelo presenta una jerarquía de herencia con User como clase base y DriverUser (atributo expedient) y TourGuideUser (atributo education) como subclases. JPA ofrece tres estrategias para mapear esta jerarquía a tablas relacionales, cada una con implicancias distintas en la base de datos y en la performance.

33. Describir las tres estrategias de mapeo de herencia. Para cada una indicar qué tablas se crean, si aparece columna discriminadora y cómo resuelve Hibernate una consulta polimórfica. Completar la tabla:

Aspecto	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Tablas creadas en la BD			
Columna discriminadora			
NULLs en columnas de subclases			
Consulta polimórfica 'todos los Users'			
Cargar un DriverUser por ID			

Aspecto	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Integridad referencial (FK posibles)			
Performance en lecturas simples (de una entidad)			
Que implica para agregar nueva subclase			

34. Implementar el mapeo de la jerarquía User/DriverUser/TourGuideUser usando la estrategia SINGLE_TABLE. Especificar @Inheritance, @DiscriminatorColumn y @DiscriminatorValue para cada clase. Incluir todos los atributos.
    v) Ejecutar los tests provistos y analizar el DDL generado: ¿cuántas tablas aparecen? ¿Qué columnas tiene la tabla? ¿Dónde están los atributos expedient y education?
    w) ¿Qué ocurre con las columnas de subclase cuando se inserta un TourGuideUser? ¿Y cuando se inserta un DriverUser?
    x) ¿Cuáles son las ventajas y desventajas de esta estrategia para este modelo concreto?
35. Reimplementar el mapeo de la misma jerarquía usando la estrategia JOINED.
    y) Ejecutar los tests y analizar el DDL: ¿cuantas tablas aparecen? ¿Qué FK existe entre ellas?
    z) Comparar el SQL generado por Hibernate al cargar un DriverUser en JOINED vs. en SINGLE_TABLE. ¿Cómo difiere?
    aa) ¿Cuáles son las ventajas y desventajas de JOINED para este modelo?
36. Realice el mismo proceso ahora con la estrategia TABLE_PER_CLASS. Indique cuál le parece la mejor estrategia para este modelo concreto y justifique su elección.
37. Route tiene relaciones muchos-a-muchos con DriverUser y TourGuideUser (subclases de User). Analizar el impacto de la estrategia de herencia sobre la tabla join:
    bb) Si la jerarquía es SINGLE_TABLE, ¿a que tabla apunta la FK en la tabla join?
    cc) Si la jerarquía es JOINED, ¿cambia la tabla destino de esa FK?
38. ¿Qué estrategia resulta más robusta ante cambios futuros como agregar una nueva subclase de User? Justificar con al menos dos argumentos.

> ***Con SINGLE_TABLE, los atributos propios de subclases (expedient, education) deben declararse nullable=true a nivel de columna aunque la lógica de negocio los requiera. Con JOINED, esa restricción puede aplicarse en la tabla de la subclase.***

Sección 3 - Repositorios, Consultas y Transacciones

Una vez mapeadas todas las entidades, esta sección aborda cómo acceder a los datos de forma organizada mediante el patrón Repository, cómo escribir consultas sobre el modelo usando HQL/JPQL, y cómo garantizar la correcta delimitación de las unidades de trabajo mediante transacciones.

3.1 Patrón de acceso a datos: DAO y Repository

39. Definir el patrón DAO (Data Access Object) y el patrón Repository. ¿Cuál es la diferencia conceptual más importante entre ambos? ¿En qué se diferencia su rol dentro de la arquitectura de la aplicación?
40. El patrón Repository puede pensarse como una colección de objetos en memoria que además persiste su estado. Describir cómo se implementa este concepto usando Hibernate: ¿qué responsabilidades concentra un repositorio? ¿Con qué objeto de Hibernate interactúa internamente?
41. Implementar un repositorio por cada entidad del modelo (PurchaseRepository, RouteRepository, UserRepository, ServiceRepository, SupplierRepository, ReviewRepository). Cada repositorio debe incluir al menos las operaciones básicas: guardar, buscar por ID, listar todos y eliminar. Utilizar la Session de Hibernate en cada implementación, tal como se ha explicado previamente utilizando el método *getCurrentSession()* sobre la *SessionFactory* (genere una inyección de este objeto tal como se muestra en el código de ejemplo proporcionado)

3.2 Transacciones

Las transacciones definen los límites de una unidad de trabajo: un conjunto de operaciones que deben ejecutarse de forma atómica, consistente, aislada y duradera.

42. ¿Que es una transacción en el contexto de Hibernate? ¿Por qué es necesaria? ¿Qué ocurre si se realizan operaciones de escritura sin una transacción activa?
43. ¿En qué capa de la aplicación debería gestionarse la transacción: en el repositorio o en la capa de servicio? Justificar la elección. ¿Qué ocurre si una misma operación necesita de varios accesos a la base de datos?
44. Implementar una capa de servicio que coordine las operaciones del modelo usando transacciones correctamente. Como mínimo implementar:
    a. Creación de todas las entidades persistentes.
    b. Actualización del precio de un servicio existente.
    c. Agregar un nuevo ítem a una compra existente.
    d. Eliminar una ruta existente siempre que no tenga compras asociadas.

3.3 Consultas con HQL y JPQL

Hibernate Query Language (HQL) es el lenguaje de consultas de Hibernate, similar a SQL pero con una diferencia fundamental. En esta sección exploraremos su uso.

45. ¿Que diferencia hay entre HQL/JPQL y SQL nativo? ¿Qué entidades, atributos y relaciones entienden HQL/JPQL que SQL no conoce directamente?
46. Implementar las siguientes consultas en los repositorios correspondientes:

a. List<Purchase> getAllPurchasesOfUsername(String username)

   Obtiene y retorna el listado de compras realizadas por el usuario con el nombre de usuario especificado por parámetro.

b. List<User> getUserSpendingMoreThan(float mount);

Obtiene y retorna el listado de usuarios que hayan efectuado alguna compra por un valor mayor o igual al pasado por parámetro.

c. List<Supplier> getTopNSuppliersInPurchases(int n);

Obtiene y retorna los n proveedores (especificado por parámetro) con mayor cantidad de apariciones en compras.

d. int getCountOfPurchasesBetweenDates (Date start, Date end);

Obtiene y retorna la cantidad de compras registradas entre dos fechas.

e. List<Route> getRoutesWithStop(Stop stop);

Obtiene y retorna el listado de rutas que incluyen una parada stop especificado por parámetro.

f. int getMaxStopOfRoutes();

Obtiene y retorna la cantidad de paradas que posee el recorrido con mayor cantidad de stops.

g. List<Route> getRoutesNotSell();

Obtiene y retorna el listado de recorridos que no fueron vendidas.

h. List<Route> getTop3RoutesWithMaxRating();

Obtiene y retorna los tres recorridos con mayor promedio de rating en las reviews de compras asociadas a dicha ruta.

i. Service getMostDemandedService();

Obtiene y retorna el servicio que más veces fue incluido en compras, teniendo en cuenta la cantidad.

j. List<TourGuideUser> getTourGuidesWithRating1();

Obtiene y retorna el listado de guías asignados a algún tour con un rating de una estrella.

47. ¿En qué casos conviene usar una consulta SQL nativa en lugar de HQL/JPQL? Describir al menos un caso concreto del modelo donde esto sería necesario.

Al completar esta práctica, el proyecto debe contar con todas las entidades mapeadas, todos los repositorios implementados con sus consultas, una capa de servicio con transacciones correctamente delimitadas.